

Managing State in React: Hinglish Explanation

Jaise-jaise React application badi hoti hai, state ko properly organize karna important ho jata hai. Agar ek hi information multiple state variables mein store kar di, ya state galat component mein rakhi, to synchronization issues aur bugs aa sakte hain.

Is chapter ka main idea hai:

UI ko directly modify mat karo. State change karo, aur React ko UI update karne do.

1. Input ke response mein state change karna

Traditional JavaScript mein hum directly UI modify kar sakte hain:

- Button disable karo
- Success message show karo
- Input hide karo
- Error message display karo

React mein approach different hai. Hum UI ke possible states describe karte hain:

- typing
- submitting
- success
- error

Phir user action ke according state change karte hain.

```
1  const [answer, setAnswer] = useState('');
2  const [error, setError] = useState(null);
3  const [status, setStatus] = useState('typing');
```

Yahan:

- answer user ka typed answer store karta hai.
- error submission error store karta hai.
- status form ki current condition batata hai.

Form submit hone ka flow

```
1  async function handleSubmit(e) {
2    e.preventDefault();
3    setStatus('submitting');
4
5    try {
```

```
6     await submitForm(answer);
7     setStatus('success');
8   } catch (err) {
9     setStatus('typing');
10    setError(err);
11  }
12 }
```

Flow kuch aisa hai:

```
1  typing → submitting → success
2                      ↘ error → typing
```

Jab status submitting hota hai, textarea aur button disable ho jate hain:

```
1  <textarea disabled={status === 'submitting'} />
2
3  <button
4    disabled={
5      answer.length === 0 ||
6      status === 'submitting'
7    }
8  >
9    Submit
10 </button>
```

Important concept

React ko yeh instruction nahi di gayi:

```
1  Button ko manually disable karo.
```

Instead, React ko condition batayi gayi:

```
1  disabled={status === 'submitting'}
```

Jab state badlegi, React automatically correct UI render karega.

2. State structure carefully choose karna

State mein sirf wahi information store karni chahiye jo separately maintain karna necessary ho.

Redundant state ki problem

Initial example mein teen state variables hain:

```
1 const [firstName, setFirstName] = useState('');
2 const [lastName, setLastName] = useState('');
3 const [fullName, setFullName] = useState('');
```

Lekin fullName already firstName aur lastName se calculate kiya ja sakta hai.

Isliye ise state mein rakhna unnecessary hai.

Problem yeh hai ki har name update par fullName ko bhi manually update karna padega:

```
1 setFirstName(e.target.value);
2 setFullName(e.target.value + ' ' + lastName);
```

Agar kisi location par setFullName() call karna bhool gaye, to state inconsistent ho jayegi.

For example:

```
1 firstName = "Ruth"
2 lastName  = "Braun"
3 fullName  = "Ruth Smith"
```

Ab data aapas mein match nahi kar raha.

Better approach

```
1 const [firstName, setFirstName] = useState('');
2 const [lastName, setLastName] = useState('');
3
4 const fullName = firstName + ' ' + lastName;
```

Ab fullName har render ke time automatically calculate hoga.

General rule

Agar koi value existing props ya state se calculate ho sakti hai, to usually use separate state mein store mat karo.

```
1 const totalPrice = price * quantity;
2 const fullName = firstName + ' ' + lastName;
3 const completedTasks = tasks.filter(task => task.done);
```

Ye derived values hain.

Golden principle

State mein duplicate ya redundant information avoid karo.

3. Components ke beech state share karna

Kabhi-kabhi do components ko synchronized rehna hota hai.

Accordion example mein do panels hain, lekin ek time par sirf ek panel active hona chahiye.

Agar har Panel apna separate state rakhe:

```
1 const [isActive, setIsActive] = useState(false);
```

to dono panels ek saath open ho sakte hain, kyunki ek component ko dusre component ki state ke baare mein nahi pata.

Solution: Lifting state up

Shared state ko individual child components se remove karke unke closest common parent mein move kar do.

```
1 export default function Accordion() {  
2   const [activeIndex, setActiveIndex] = useState(0);  
3 }
```

Parent decide karta hai kaunsa panel active hai:

```
1 <Panel  
2   title="About"  
3   isActive={activeIndex === 0}  
4   onShow={() => setActiveIndex(0)}  
5 />  
6  
7 <Panel  
8   title="Etymology"  
9   isActive={activeIndex === 1}  
10  onShow={() => setActiveIndex(1)}  
11 />
```

Child component ko do important props mil rahe hain:

```
1 function Panel({  
2   title,  
3   children,  
4   isActive,  
5   onShow  
6 }) {  
7   // ...  
8 }
```

- isActive batata hai panel open hai ya nahi.
- onShow parent ki state update karta hai.

Child button click hone par:

```
1 <button onClick={onShow}>
```

```
2   Show
3   </button>
```

Data flow

```
1  Accordion state
2      ↓ props
3      Panel
4      ↓ event callback
5  Accordion state update
```

React mein data normally parent se child ki taraf flow karta hai.

Kab state lift karni chahiye?

Jab:

- Multiple components ko same information chahiye.
 - Components ko synchronized rehna ho.
 - Ek component ka change dusre ko affect karta ho.
 - Parent ko children ki condition control karni ho.
-

4. State ko preserve aur reset karna

React state component ki JSX mein position se connected hoti hai.

Agar same component same position par render ho raha hai, React generally uski existing state preserve karta hai.

Chat example mein recipient change hota hai:

```
1  <Chat contact={to} />
```

Lekin component type aur position same hai. Sirf contact prop change hui hai.

Isliye Chat component ke andar ka text state preserve hota rahega:

```
1  const [text, setText] = useState('');
```

Suppose user ne Taylor ke liye message type kiya, lekin send nahi kiya. Ab user Alice select karta hai. Input ka old message abhi bhi present reh sakta hai.

Isse accidentally wrong person ko message send ho sakta hai.

State reset karne ke liye key

```
1  <Chat key={to.email} contact={to} />
```

Jab recipient change hoga, key bhi change hogi.

```
1 Taylor → key="taylor@mail.com"
2 Alice → key="alice@mail.com"
```

React different key ko different component identity maanta hai. Purana component remove hota hai aur naya component fresh state ke saath create hota hai.

Isliye:

```
1 const [text, setText] = useState('');
```

dobara initial empty string se start karega.

Important distinction

key sirf lists ke liye nahi hoti. Iska use component identity control karne aur state reset karne ke liye bhi ho sakta hai.

Simple rule

- Same component, same position, same key → state preserve
 - Different component type → state reset
 - Different key → state reset
-

5. Complex state logic ko reducer mein extract karna

Small component mein kuch useState calls manageable hoti hain. Lekin jab:

- Bahut saare state updates hon
- Multiple event handlers hon
- State update rules complex hon
- Different actions same data ko modify kar rahi hon

tab logic scattered aur difficult to maintain ho sakta hai.

Is situation mein useReducer helpful hai.

```
1 const [tasks, dispatch] = useReducer(
2   tasksReducer,
3   initialTasks
4 );
```

useReducer teen important cheezon ke saath kaam karta hai:

1. Current state: tasks
2. Action bhejne wala function: dispatch
3. State update logic: tasksReducer

Action dispatch karna

Task add karne ke liye:

```
1 dispatch({
2   type: 'added',
3   id: nextId++,
4   text: text,
5 });
```

Task change karne ke liye:

```
1 dispatch({
2   type: 'changed',
3   task: task
4 });
```

Task delete karne ke liye:

```
1 dispatch({
2   type: 'deleted',
3   id: taskId
4 });
```

Event handler directly state modify nahi karta. Woh sirf batata hai:

User ne kya action perform kiya?

Reducer ka role

```
1 function tasksReducer(tasks, action) {
2   switch (action.type) {
3     case 'added': {
4       return [
5         ...tasks,
6         {
7           id: action.id,
8           text: action.text,
9           done: false
10        }
11      ];
12    }
13
14    case 'changed': {
15      return tasks.map(t => {
16        if (t.id === action.task.id) {
17          return action.task;
```

```

18     }
19     return t;
20   });
21 }
22
23 case 'deleted': {
24   return tasks.filter(
25     t => t.id !== action.id
26   );
27 }
28
29 default: {
30   throw Error(
31     'Unknown action: ' + action.type
32   );
33 }
34 }
35 }

```

Reducer decide karta hai ki action ke according next state kya hogi.

Flow

```

1  User event
2    ↓
3  dispatch(action)
4    ↓
5  reducer(currentState, action)
6    ↓
7  new state
8    ↓
9  React re-render

```

Reducer ke benefits

- Saari state update logic ek jagah milti hai.
- Event handlers concise ho jate hain.
- Complex changes follow karna easier hota hai.
- State transitions predictable hoti hain.
- Update logic component UI se separate ho jati hai.

Reducer ka important rule

Reducer ko existing state mutate nahi karni chahiye.

Wrong:

```
1 tasks.push(newTask);  
2 return tasks;
```

Correct:

```
1 return [...tasks, newTask];
```

Update ke liye:

```
1 return tasks.map(task =>  
2   task.id === updatedTask.id  
3     ? updatedTask  
4     : task  
5 );
```

Delete ke liye:

```
1 return tasks.filter(task =>  
2   task.id !== deletedId  
3 );
```

6. Context se deeply data pass karna

Normally parent child ko props se data deta hai:

```
1 <Child theme={theme} />
```

Agar deeply nested component ko data chahiye, to same prop beech ke multiple components se forward karna pad sakta hai:

```
1 App  
2   ↓ theme  
3   Layout  
4     ↓ theme  
5     Sidebar  
6       ↓ theme  
7       Button
```

Ho sakta hai Layout aur Sidebar khud theme use na kar rahe hon, lekin unhe prop forward karni pad rahi hai.

Is problem ko commonly **prop drilling** kaha jata hai.

Context solution

Context parent ko information provide karne deta hai, jise tree ke andar koi bhi nested component directly read kar sakta hai.

Step 1: Context create karo

```
1 import { createContext } from 'react';  
2  
3 export const LevelContext = createContext(0);
```

0 default value hai.

Step 2: Context provide karo

```
1 <LevelContext value={level + 1}>  
2   {children}  
3 </LevelContext>
```

Provider ke andar ke components is value ko access kar sakte hain.

Step 3: Context read karo

```
1 const level = useContext(LevelContext);
```

Heading component level ke according correct HTML heading return karta hai:

```
1 switch (level) {  
2   case 1:  
3     return <h1>{children}</h1>;  
4   case 2:  
5     return <h2>{children}</h2>;  
6   case 3:  
7     return <h3>{children}</h3>;  
8 }
```

Nested Section closest parent context ka level read karta hai aur usmein 1 add karke children ko provide karta hai:

```
1 const level = useContext(LevelContext);  
2  
3 <LevelContext value={level + 1}>  
4   {children}  
5 </LevelContext>
```

Result kuch aisa hoga:

```
1 Section level 1 → h1  
2   Section level 2 → h2  
3     Section level 3 → h3  
4       Section level 4 → h4
```

Context kab useful hai?

- Theme
- Logged-in user

- Language or locale
- Authentication information
- Application settings
- Deeply shared state
- Complex screen ka shared data

Context ko unnecessarily use mat karo

Agar data sirf direct child ko chahiye, normal props usually simpler aur clearer hain.

Context tab useful hai jab data genuinely tree mein deeply ya bahut saare components ko chahiye.

7. Reducer aur Context ko combine karna

useReducer aur Context different problems solve karte hain:

Feature	Purpose
Reducer	State kaise update hogi
Context	State aur dispatch components tak kaise pahunchenge

In dono ko combine karke complex screens ka state management scalable banaya ja sakta hai.

Separate contexts

Example mein do contexts create kiye gaye hain:

```
1  const TasksContext = createContext(null);
2  const TasksDispatchContext = createContext(null);
```

- TasksContext current tasks provide karta hai.
- TasksDispatchContext dispatch function provide karta hai.

Provider component

```
1  export function TasksProvider({ children }) {
2    const [tasks, dispatch] = useReducer(
3      tasksReducer,
4      initialTasks
5    );
6
7    return (
8      <TasksContext value={tasks}>
9        <TasksDispatchContext value={dispatch}>
```

```
10      {children}
11      </TasksDispatchContext>
12    </TasksContext>
13  );
14 }
```

Ab TasksProvider ke andar kisi bhi depth par component tasks read ya update kar sakta hai.

Application ko provider se wrap kiya gaya hai:

```
1 <TasksProvider>
2   <h1>Day off in Kyoto</h1>
3   <AddTask />
4   <TaskList />
5 </TasksProvider>
```

Custom hooks

Directly har component mein useContext likhne ke bajaye reusable hooks banaye gaye hain:

```
1 export function useTasks() {
2   return useContext(TasksContext);
3 }
4
5 export function useTasksDispatch() {
6   return useContext(TasksDispatchContext);
7 }
```

Ab component mein code clean dikhta hai:

```
1 const tasks = useTasks();
2 const dispatch = useTasksDispatch();
```

AddTask component

```
1 const dispatch = useTasksDispatch();
```

Task add karne ke liye:

```
1 dispatch({
2   type: 'added',
3   id: nextId++,
4   text: text,
5 });
```

AddTask ko parent se onAddTask prop lene ki zarurat nahi hai.

TaskList component

```
1 const tasks = useTasks();
```

Tasks directly context se mil rahe hain:

```
1 {tasks.map(task => (  
2   <li key={task.id}>  
3     <Task task={task} />  
4   </li>  
5 )}}
```

Nested Task component bhi dispatch access kar sakta hai:

```
1 const dispatch = useTasksDispatch();
```

Isliye intermediate components ko unnecessary callbacks forward nahi karne padte.

Complete mental model

Managing State chapter ko ek flow ke through samjho:

Step 1: UI states identify karo

Socho UI kin conditions mein ho sakti hai:

```
1 typing  
2 submitting  
3 success  
4 error
```

Step 2: Minimum state store karo

Jo value calculate ho sakti hai, usko separate state mat banao:

```
1 const fullName = firstName + ' ' + lastName;
```

Step 3: State sahi owner mein rakho

Agar multiple components ko same state chahiye, use closest common parent mein lift karo.

```
1 Parent owns state  
2 Children receive props  
3 Children trigger callbacks
```

Step 4: Component identity samjho

React state JSX tree ki position aur identity se associate hoti hai.

State reset karni ho to different key de sakte ho:

```
1 <Chat key={to.email} contact={to} />
```

Step 5: Complex updates ko reducer mein rakho

```
1 Event → dispatch action → reducer → new state
```

Step 6: Deep data sharing ke liye context use karo

```
1 Provider → deeply nested consumer
```

Step 7: Large screen ke liye reducer + context

```
1 Reducer manages updates  
2 Context distributes state and dispatch
```

Practical decision guide

useState use karo jab:

- State simple ho
- Updates straightforward hon
- State ek component ya nearby children tak limited ho

```
1 const [text, setText] = useState('');
```

State lift karo jab:

- Sibling components ko synchronize karna ho
- Multiple children ko same state chahiye
- Parent ko state control karni ho

key use karo jab:

- Same position par rendered component ki state intentionally reset karni ho
- Different record ya entity ko fresh component treat karna ho

```
1 <Form key={user.id} user={user} />
```

useReducer use karo jab:

- State update logic complex ho
- Bahut saare related actions hon
- Multiple handlers same state update kar rahe hon
- Update logic centralized rakhni ho

Context use karo jab:

- Data deeply nested components tak chahiye
- Bahut saare components same information consume karte hon

- Prop drilling inconvenient ho gayi ho

Reducer + Context use karo jab:

- Complex screen ka shared state ho
 - Distant components ko state read aur update karni ho
 - Update logic ko centralized rakhna ho
-

Most important takeaways

1. **React mein UI directly manipulate nahi karte.** State change karte hain, UI automatically update hoti hai.
2. **Duplicate state bugs create karti hai.** Derived values ko render ke time calculate karo.
3. **Shared state ka ek clear owner hona chahiye.** Sibling components ke liye state common parent mein lift karo.
4. **State component ki identity se connected hoti hai.** Different key dekar state reset kar sakte ho.
5. **Reducer complex state updates ko centralize karta hai.** Components action describe karte hain, reducer update perform karta hai.
6. **Context prop drilling reduce karta hai.** Lekin simple parent-child communication ke liye props better ho sakte hain.
7. **Reducer + Context ek scalable pattern hai.** Reducer update logic sambhalta hai aur Context state plus dispatch ko deep components tak pahunchata hai.

Explanation approach: Maine provided chapter ko same order mein simplify kiya hai, har concept ka purpose, code flow aur practical use case clarify kiya hai. Koi external resource add nahi kiya gaya.